



```
In [1]: # --- Step 1: Import Required Libraries ---
# (from Experiment 7, Page 2)
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.datasets import fetch_california_housing # Added to load a runnab

# --- Step 2: Load Dataset ---
# Using California Housing dataset as a replacement for 'your_dataset.csv'
housing = fetch_california_housing()
data = pd.DataFrame(housing.data, columns=housing.feature_names)
data['PRICE'] = housing.target # Using 'PRICE' as the target column

print("--- Dataset Loaded (California Housing) ---")
print(data.head())
print("\n")

# --- Step 3: Separate Features and Target ---
# (from Experiment 7, Page 2)
X = data.drop('PRICE', axis=1)
y = data['PRICE']

# --- Step 4: Split Data into Training and Testing Sets ---
# (from Experiment 7, Page 2)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- Step 5: Feature Scaling ---
# (from Experiment 7, Page 3)
# Scaling X features
scaler_X = StandardScaler()
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

# Scaling y target (needed for SVR)
scaler_y = StandardScaler()
# Reshape y_train to 2D array for scaling, then flatten back with .ravel()
y_train_scaled = scaler_y.fit_transform(y_train.values.reshape(-1, 1)).ravel()

# --- Step 6: Define Models ---
# (from Experiment 7, Page 3)
models = {
    "Linear Regression": LinearRegression(),
    # Polynomial Regression uses a LinearRegression model on transformed features
    "Polynomial Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
    "SVR": SVR(kernel='rbf')
```

```

}

# --- Step 7: Polynomial Feature Transformation ---
# (from Experiment 7, Page 3)
# Create 2nd-degree polynomial features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly_train = poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

# --- Step 8: Train and Evaluate Models ---
# (from Experiment 7, Page 3-4)
results = {}

print("--- Training and Evaluating Models ---")

for name, model in models.items():
    if name == "Polynomial Regression":
        # Train on polynomial features
        model.fit(X_poly_train, y_train)
        y_pred = model.predict(X_poly_test)

    elif name == "SVR":
        # Train on scaled X and scaled y
        model.fit(X_train_scaled, y_train_scaled)
        # Predict on scaled X, then inverse_transform the scaled predictions
        y_pred_scaled = model.predict(X_test_scaled)
        y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()

    else:
        # Train on original (unscaled) X for tree-based models
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    # Store evaluation metrics
    results[name] = {
        "MAE": mean_absolute_error(y_test, y_pred),
        "MSE": mean_squared_error(y_test, y_pred),
        "R2_Score": r2_score(y_test, y_pred)
    }
    print(f"Completed: {name}")

# --- Step 9: Display Results ---
# (from Experiment 7, Page 4)
results_df = pd.DataFrame(results).T
results_df = results_df.sort_values(by="R2_Score", ascending=False) # Sort by

print("\n--- Model Comparison Results ---")
print(results_df)
print("\n--- Conclusion ---")
print(f"The best performing model is: {results_df.index[0]} (R2 Score: {result

```

--- Dataset Loaded (California Housing) ---

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	PRICE
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

--- Training and Evaluating Models ---

Completed: Linear Regression

Completed: Polynomial Regression

Completed: Decision Tree

Completed: Random Forest

Completed: SVR

--- Model Comparison Results ---

	MAE	MSE	R2_Score
Random Forest	0.327543	0.255368	0.805123
SVR	0.397283	0.354151	0.729740
Polynomial Regression	0.467001	0.464302	0.645682
Decision Tree	0.454679	0.495235	0.622076
Linear Regression	0.533200	0.555892	0.575788

--- Conclusion ---

The best performing model is: Random Forest (R2 Score: 0.8051)

In []:

In []:

In []: